```
class MaxHeap:
    def __init__(self):
        self.heap = []

    # Insert an element
    # Time: O(log n)
    # Space: O(1) extra
    def insert(self, value):
        self.heap.append(value)
        self._heapify_up(len(self.heap) - 1)

    # Move element upward
    def _heapify_up(self, index):
        while index > 0:
            parent = (index - 1) // 2

            if self.heap[index] > self.heap[parent]:
                self.heap[index], self.heap[parent] = \
                    self.heap[parent], self.heap[index]
                index = parent
            else:
                break

    # Remove maximum element
    # Time: O(log n)
    # Space: O(1) extra
    def extract_max(self):
        if not self.heap:
            return None

        if len(self.heap) == 1:
            return self.heap.pop()

        max_value = self.heap[0]

        self.heap[0] = self.heap.pop()
        self._heapify_down(0)

        return max_value

    # Move element downward
    def _heapify_down(self, index):
        n = len(self.heap)

        while True:
            largest = index
            left = 2 * index + 1
            right = 2 * index + 2
```

```
            if left < n and self.heap[left] > self.heap[largest]:
                largest = left

            if right < n and self.heap[right] > self.heap[largest]:
                largest = right

            if largest == index:
                break

            self.heap[index], self.heap[largest] = \
                self.heap[largest], self.heap[index]

            index = largest

    # Get maximum element
    # Time: O(1)
    # Space: O(1)
    def get_max(self):
        return self.heap[0] if self.heap else None


# Example
max_heap = MaxHeap()

max_heap.insert(10)
max_heap.insert(20)
max_heap.insert(5)
max_heap.insert(30)
max_heap.insert(15)

print("Max Heap:", max_heap.heap)
print("Maximum:", max_heap.get_max())
print("Removed:", max_heap.extract_max())
print("After removal:", max_heap.heap)
```

```
Max Heap: [30, 20, 5, 10, 15]
Maximum: 30
Removed: 30
After removal: [20, 15, 5, 10]
```

```
class MinHeap:
    def __init__(self):
        self.heap = []

    # Insert an element
    # Time: O(log n)
    # Space: O(1) extra
    def insert(self, value):
        self.heap.append(value)
        self._heapify_up(len(self.heap) - 1)
```

```
        self._heapify_up(len(self.heap) - 1)

    # Move element upward
    def _heapify_up(self, index):
        while index > 0:
            parent = (index - 1) // 2

            if self.heap[index] < self.heap[parent]:
                self.heap[index], self.heap[parent] = \
                    self.heap[parent], self.heap[index]
                index = parent
            else:
                break

    # Remove minimum element
    # Time: O(log n)
    # Space: O(1) extra
    def extract_min(self):
        if not self.heap:
            return None

        if len(self.heap) == 1:
            return self.heap.pop()

        min_value = self.heap[0]

        self.heap[0] = self.heap.pop()
        self._heapify_down(0)

        return min_value

    # Move element downward
    def _heapify_down(self, index):
        n = len(self.heap)

        while True:
            smallest = index
            left = 2 * index + 1
            right = 2 * index + 2

            if left < n and self.heap[left] < self.heap[smallest]:
                smallest = left

            if right < n and self.heap[right] < self.heap[smallest]:
                smallest = right

            if smallest == index:
                break

            self.heap[index], self.heap[smallest] = \
                self.heap[smallest], self.heap[index]
```

```
            index = smallest

    # Get minimum element
    # Time: O(1)
    # Space: O(1)
    def get_min(self):
        return self.heap[0] if self.heap else None


# Example
min_heap = MinHeap()

min_heap.insert(10)
min_heap.insert(20)
min_heap.insert(5)
min_heap.insert(30)
min_heap.insert(15)

print("Min Heap:", min_heap.heap)
print("Minimum:", min_heap.get_min())
print("Removed:", min_heap.extract_min())
print("After removal:", min_heap.heap)
```

```
Min Heap: [5, 15, 10, 30, 20]
Minimum: 5
Removed: 5
After removal: [10, 15, 20, 30]
```